

Capacity AR

training-module-design.md

Diseño del módulo de entrenamiento

Diseño técnico del módulo de capacitación del Equipo 2. Cubre el motor 5P, la integración con IA, el sistema de tutores, alertas y métricas individuales.

Vision General

El módulo de capacitación es la pieza central del sistema. Recibe un GapReport del Equipo 1 y construye una experiencia de aprendizaje adaptativa para el estudiante, basada en el marco pedagógico de las 5 P y supervisada por un tutor humano.

GapReport JSON -> Learning Path -> 5P + IA + Tutor -> Estudiante listo

Entrada

Definida por el Equipo 1 en `gap-engine-mvp.md`. Resumen:

```
{
  "student": {...},
  "company": {...},
  "target_role": {...},
  "readiness_score": 45,
  "skills": [
    { "name": "Git", "gap_level": 2, "priority": "HIGH", "status": "MISSING", ... },
    { "name": "SQL", "gap_level": 1, "priority": "HIGH", "status": "NEEDS_WORK", ... }
  ]
}
```

Salida

Un learning path con:

- Módulos (1 por habilidad faltante).
- Unidades dentro de cada módulo, alineadas a las fases 5P.
- Ejercicios generados o validados con IA.
- Orden sugerido según prioridad y dependencias.
- Tutor asignado.
- Criterios de avance (umbral 80%).

Marco Pedagogico - Las 5 P

Fase	Que hace	Salida
Pasion	Anclaje motivacional, conecta habilidad con la empresa elegida	Mensaje contextualizado
Play	Sandbox, micro-leccion 5-10 min, exploracion sin penalizacion	Unidad interactiva
Practica	Ejercicios adaptativos, umbral de dominio 80%	Evaluacion formativa
Paciencia	Detecta frustracion, normaliza el error	Mensaje empatico
Perseverancia	Visualiza progreso, micro-logros	Hito visible

Paciencia y Perseverancia operan transversalmente (no son fases separadas).

Sub-modulos

```
app/modules/  
  learning_paths/      # path completo del estudiante  
  learning_content/    # unidades, ejercicios, integracion Gemini  
  attempts/           # intentos + logica dominio 80%  
  sessions/           # registro de sesiones  
  tutors/             # padron de tutores  
  tutor_assignments/  # asignacion estudiante-tutor  
  alerts/             # alertas internas  
  validations/        # entregables que requieren tutor humano  
  student_metrics/    # metricas individuales (alumno y tutor)
```

Cada sub-modulo respeta el patron de `users/` : `router.py` , `service.py` , `repository.py` , `models.py` , `schemas.py` .

Modelo De Datos Propuesto

learning_paths

```
id  
gap_analysis_id      (FK a gap_analyses del Eq1)  
student_id           (FK)  
status              (ACTIVE | COMPLETED | PAUSED | ABANDONED)  
readiness_target     (score objetivo, default 80)  
created_at  
updated_at
```

learning_modules

Un modulo por habilidad faltante del gap.

```
id
learning_path_id      (FK)
skill_name            (Git, SQL, etc.)
priority              (HIGH | MEDIUM | LOW)
order_index
mastery_threshold     (default 0.80)
current_mastery       (0.0 a 1.0)
status               (PENDING | IN_PROGRESS | MASTERED)
```

learning_units

Lecciones dentro de un modulo, alineadas a 5P.

```
id
module_id             (FK)
phase                 (PACION | PLAY | PRACTICA)
title
content_json          (texto, ejemplos, generado por IA)
order_index
estimated_minutes
```

exercises

```
id
unit_id              (FK)
prompt
type                 (MULTIPLE_CHOICE | TEXT | CODE) - a definir con equipo
expected_answer
difficulty           (1 a 5)
```

attempts

```
id
exercise_id          (FK)
student_id           (FK)
unit_id              (FK)
skill_name
answer
is_correct
score                (0.0 a 1.0)
retries
time_spent_seconds
ai_feedback          (generado por Gemini)
created_at
```

sessions

```
id
student_id          (FK)
learning_path_id    (FK)
started_at
ended_at
duration_seconds
```

skill_mastery

Estado actual de dominio del estudiante por habilidad.

```
id
student_id          (FK)
skill_name
current_level       (0 a 5)
target_level
mastery_percentage  (0.0 a 1.0)
last_practiced_at
```

events

Tabla append-only que sirve al Equipo 1 para sus metricas globales.

```
id
student_id
type                (session_started | exercise_completed | mastery_achieved |
etc.)
payload_json
created_at
```

tutors

```
id
user_id             (FK a users)
specialties         (lista de skills, ej: ["Git", "Python"])
max_students
available
```

tutor_assignments

```
id
tutor_id            (FK)
student_id          (FK)
learning_path_id    (FK)
assigned_at
unassigned_at       (nullable)
```

alerts

```
id
learning_path_id      (FK)
student_id            (FK)
type                  (LOW_PERSEVERANCE | ABANDON_RISK | LOST_PASSION |
ANOMALY_PATIENCE)
level                 (1 a 3)
payload_json          (variables que dispararon la alerta)
resolved
resolved_by_tutor_id  (FK nullable)
created_at
resolved_at           (nullable)
```

validations

Entregables que requieren ojo humano.

```
id
learning_path_id      (FK)
student_id            (FK)
unit_id               (FK)
type                  (PRESENTATION | CODE_REVIEW | PROJECT)
content_url            (link al entregable)
status                (PENDING | APPROVED | REJECTED | NEEDS_REVISION)
tutor_feedback
validated_by_tutor_id (FK nullable)
submitted_at
validated_at          (nullable)
```

Endpoints

Learning Path

```
POST  /learning-paths      (crea desde un GapReport)
GET    /learning-paths/{id}
GET    /students/{id}/learning-path (path activo del estudiante)
GET    /learning-paths/{id}/next-unit (que toca hacer ahora)
PATCH /learning-paths/{id} (tutor o admin pueden ajustar)
```

Contenido y ejercicios

```
GET    /units/{id}
GET    /units/{id}/exercises
POST   /attempts           (registra intento, IA evalua)
GET    /attempts/{id}
GET    /students/{id}/attempts (historial)
```

Sesiones

```
POST /sessions/start
POST /sessions/{id}/end
GET /students/{id}/sessions
```

Tutores

```
POST /tutors
GET /tutors
GET /tutors/{id}
PATCH /tutors/{id}
POST /tutor-assignments (asigna tutor a estudiante)
GET /tutors/{id}/dashboard (panel del tutor)
GET /tutors/{id}/students (asignados)
```

Alertas

```
GET /alerts (filtrable por tutor, estudiante, estado)
GET /alerts/{id}
PATCH /alerts/{id}/resolve
```

Validaciones

```
POST /validations (estudiante sube entregable)
GET /validations/{id}
PATCH /validations/{id}/validate (tutor aprueba o rechaza)
GET /tutors/{id}/pending-validations
```

Métricas individuales

```
GET /students/{id}/progress (motivacional - para el alumno)
GET /students/{id}/metrics (de seguimiento - para el tutor)
```

Ciclo Pedagógico Detallado

Generación del path

1. Recibe GapReport.
2. Por cada skill con status MISSING o NEEDS_WORK:
 - Crea un learning_module.
3. Ordena módulos por priority (HIGH > MEDIUM > LOW) y por dependencias.

4. Por cada modulo:
 - Genera 3 unidades base: Pasion, Play, Practica.
 - Cada unidad se llena con contenido via Gemini.
5. Persiste todo.
6. Asigna tutor (algoritmo a definir).
7. Devuelve learning_path_id al estudiante.

Ejecucion de una unidad

1. Estudiante hace GET /learning-paths/{id}/next-unit.
2. Sistema entrega la siguiente unidad pendiente.
3. Estudiante consume el contenido (texto, video, sandbox).
4. Para unidad de PRACTICA:
 - Estudiante intenta los ejercicios.
 - Cada intento dispara POST /attempts.
 - Gemini evalua el intento y devuelve ai_feedback.
 - Se actualiza skill_mastery.
5. Cuando mastery ≥ 0.80 :
 - El modulo pasa a status MASTERED.
 - Se emite evento mastery_achieved.
 - Se desbloquea el siguiente modulo.
6. Si mastery < 0.80 despues de N intentos:
 - Se cicla a una explicacion alternativa (no la misma).
 - Se refuerza con mensaje de Paciencia/Perseverancia.

Deteccion de alertas

Reglas iniciales (a refinar):

```
LOW_PERSEVERANCE
-> 5 intentos fallidos seguidos en el mismo ejercicio
-> nivel 1

ABANDON_RISK
-> 72h sin actividad
-> nivel 2
-> 7 dias sin actividad
-> nivel 3 (escalar a admin)

LOST_PASSION
-> tiempo promedio de sesion cae mas de 50% en 5 dias
-> nivel 1

ANOMALY_PATIENCE
-> lenguaje agresivo detectado por NLP en chat con el coach
-> nivel 2
```

Cada alerta queda persistida y se notifica al tutor asignado.

Integracion Con IA - Google Gemini

Por que Gemini

- Gratis con cuenta Google (sin tarjeta).
- 1M tokens/dia (suficiente para TP universitario).
- 15 req/min en plan free.
- SDK Python oficial: `google-generativeai`.
- Modelo: `gemini-1.5-flash` (rapido y barato).

Casos de uso

1. Generar contenido de unidades (Pasion, Play, Practica) contextualizado al perfil del estudiante.
2. Evaluar respuestas de ejercicios.
3. Generar feedback formativo en cada intento.
4. Reformular explicaciones alternativas cuando el estudiante no alcanza el dominio.
5. Detectar tono emocional en mensajes (Paciencia).

Configuracion

Variable de entorno:

```
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-1.5-flash
```

Cliente envuelto en `app/shared/llm.py` para poder cambiar el proveedor sin tocar la logica de negocio.

Fallback

Si Gemini falla o esta sin cuota:

- Logear el error.
- Usar contenido estatico precargado por skill (un fallback minimo).
- Notificar al admin.

Metricas Individuales

Para el estudiante (motivacional)

Metrica	Como se muestra
% completado del path	"Vas por el 40%"
Habilidades dominadas	"Dominaste Git, te faltan SQL y HTTP APIs"
Streak de dias activos	"7 dias seguidos aprendiendo"
Tiempo invertido	"12 horas de practica"
Proximo hito	"Te faltan 3 ejercicios para dominar SQL"

Importante: no se muestra el `readiness_score` numerico crudo.

Para el tutor (seguimiento)

Metrica	Para que
Frecuencia de sesiones	Detectar caida de actividad
Duracion promedio de sesion	Detectar sesiones cada vez mas cortas
Dias de inactividad	Disparar alerta de abandono
Tasa de exito en ejercicios	Detectar frustracion
Reintentos por unidad	Detectar bloqueo
Ratio teoria/practica	Detectar pasividad
Habilidades estancadas	Saber donde reforzar manualmente

Eventos Emitidos Al Equipo 1

El Equipo 1 calcula metricas globales leyendo eventos. Lista inicial:

```
session_started
session_ended
exercise_attempted
exercise_completed
unit_completed
module_completed
path_completed
mastery_achieved
alert_raised
tutor_intervened
validation_submitted
validation_approved
```

Cada evento se persiste en la tabla `events`. El Equipo 1 puede leerla directamente o se expone un endpoint `GET /events` (a decidir).

Decisiones Pendientes

1. Tipo de respuesta del estudiante: texto libre, multiple opcion o codigo.
2. Orden de fases 5P: fijo o decidido por IA.
3. Algoritmo de asignacion de tutor: manual, round-robin, por especialidad.
4. Cantidad de intentos antes de cambiar a explicacion alternativa.
5. Umbral exacto para cada nivel de alerta (los listados son una propuesta inicial).
6. Como se entrega el contenido inicial (texto plano via API, markdown, HTML).

Aplicacion De La Matriz De Accesos

Ver [access-matrix.md](#) . Resumen aplicado al modulo:

- Estudiante: solo lo propio, sin ver score crudo, sin ver alertas.
- Tutor: solo sus asignados, ve alertas y valida entregables.
- Empresa: para MVP no ve nada del avance. Solo carga ofertas.
- Admin: todo.

Estado MVP

Pieza	Estado
Modelo de datos	Disenado, sin migrar
Endpoints	Disenados, sin implementar
Integracion Gemini	Decidida, sin codear
Sin login	OK para MVP, mock con headers
Contrato con Eq1	OK (GapReport definido)
Eventos hacia Eq1	Por acordar